

Chapter 2: Sets and Tuples in Python

Sets

A **set** in Python is a collection used to store multiple values together. The most important feature of a set is that it stores **only unique elements**, which means duplicate values are not allowed. If we try to put the same value into a set more than once, Python keeps only one copy of that value. Sets are commonly useful when we have a collection of data and want to remove duplicate values or perform operations such as combining two groups, finding common elements, or finding elements that are different.

Sets are generally written using **curly brackets { }**. For example, if we want to create a set containing the numbers 10, 20, 30, and 40, we can write:

```
numbers = {10, 20, 30, 40}
```

```
print(numbers)
```

The `numbers` variable stores the set. The `print()` statement displays all the elements in the set. The output may look like this:

```
{40, 10, 20, 30}
```

You may notice that the order is different from the order in which we entered the values. This is because a **set is unordered**. We should not depend on a particular order when working with a set.

A set can also be created using the `set()` function. For example:

```
numbers = set([10, 20, 30, 40])
```

```
print(numbers)
```

Here, `[10, 20, 30, 40]` is first written as a list, and the `set()` function converts those values into a set.

Characteristics of Sets

One important characteristic of a set is **uniqueness**. This means that a set does not keep duplicate values. For example:

```
numbers = {10, 20, 20, 30, 30, 30}
```

```
print(numbers)
```

The output will be:

{10, 20, 30}

Even though we entered 20 twice and 30 three times, the set stores each value only once. Python automatically removes the duplicate values when creating the set. This is one of the main reasons why sets are useful. If a student list contains the same name multiple times and we want only the unique names, a set can help us.

For example:

```
names = {"Ravi", "Sita", "Ravi", "John", "Sita"}
```

```
print(names)
```

The output will contain only the unique names:

```
{"Ravi", "Sita", "John"}
```

The two occurrences of "Ravi" become one "Ravi", and the two occurrences of "Sita" become one "Sita".

Another important characteristic is that **sets are unordered**. When we create a set such as:

```
fruits = {"Apple", "Banana", "Mango"}
```

```
print(fruits)
```

we should not expect Python to always display the values in exactly this order:

Apple

Banana

Mango

The order can be different because sets do not maintain elements according to their position.

Sets are also **mutable**. Mutable means that we can change the contents of the set after creating it. We can add new elements or remove existing elements.

For example:

```
numbers = {10, 20, 30}
```

```
numbers.add(40)
```

```
print(numbers)
```

Initially, the set contains:

```
{10, 20, 30}
```

The statement `numbers.add(40)` adds the value 40 to the set. After adding it, the set contains four values:

```
{10, 20, 30, 40}
```

However, even though we added 40, we cannot say that 40 is at position 3. This is because sets do not use index positions.

Sets Do Not Support Indexing

This is an important point for beginners. In a list or tuple, every element has an index. For example:

```
numbers = (10, 20, 30)
```

Here we can think of the values as:

10 → index 0

20 → index 1

30 → index 2

Therefore, we can write:

```
print(numbers[0])
```

and Python gives:

```
10
```

But a set does not work this way.

Consider:

```
numbers = {10, 20, 30}
```

```
print(numbers[0])
```

Here, `numbers[0]` means “give me the element at index 0.” But the set does not have index positions such as 0, 1, and 2. Therefore, Python gives an error.

The important thing to remember is that a set knows **which values it contains**, but it does not provide a fixed position for each value.

If we want to access the elements of a set, we can use a `for` loop:

```
numbers = {10, 20, 30}
```

```
for number in numbers:  
    print(number)
```

The loop takes one value from the set at a time and stores it temporarily in the variable `number`. The values are then printed. The order in which they appear should not be assumed to be fixed.

Set Operations

Set operations are used when we want to compare or combine two sets. They are particularly useful when working with groups of data. The four important set operations are **Union, Intersection, Difference, and Symmetric Difference**.

Let us use these two sets throughout the examples:

$A = \{1, 2, 3, 4, 5\}$

$B = \{4, 5, 6, 7, 8\}$

Here, A contains 1, 2, 3, 4, 5, while B contains 4, 5, 6, 7, 8. The values 4 and 5 are common to both sets.

Union

Union means combining the elements of two sets. When we perform a union, all the elements from both sets are included, but duplicate values are kept only once.

The union operator in Python is `|`.

$A = \{1, 2, 3, 4, 5\}$

$B = \{4, 5, 6, 7, 8\}$

```
result = A | B
```

```
print(result)
```

The result is:

$\{1, 2, 3, 4, 5, 6, 7, 8\}$

Let us understand what happened. Set A has 1, 2, 3, 4, 5, and set B has 4, 5, 6, 7, 8. Python takes all values from both sets. Since 4 and 5 occur in both sets, they are not repeated. Therefore, the final set contains 1, 2, 3, 4, 5, 6, 7, 8.

We can also perform union using the `union()` method:

```
result = A.union(B)
```

```
print(result)
```

Both `A | B` and `A.union(B)` perform the same basic operation.

A real-life example makes union easier to understand. Suppose some students joined cricket and some students joined football.

```
cricket = {"Ravi", "Sita", "John"}
```

```
football = {"John", "Meena", "Ravi"}
```

students = cricket | football

print(students)

The result contains:

{"Ravi", "Sita", "John", "Meena"}

Here, we want **every student who joined either cricket or football**. Ravi and John appear in both groups, but they are listed only once because a set does not allow duplicates.

So, remember **Union = combine everything**.

Intersection

Intersection means finding the elements that are common to both sets. In other words, we ask: **“Which values are present in both sets?”**

The intersection operator is **&**.

A = {1, 2, 3, 4, 5}

B = {4, 5, 6, 7, 8}

result = A & B

print(result)

The output is:

{4, 5}

Why do we get only 4 and 5? Let us compare the sets. 1 is only in A, so it is not included. 2 is only in A, so it is not included. 3 is only in A, so it is not included. 4 is in both A and B, so it is included. 5 is also in both, so it is included. 6, 7, and 8 are only in B, so they are not included.

Therefore:

A = {1, 2, 3, 4, 5}

B = {4, 5, 6, 7, 8}

Common = {4, 5}

We can also use the **intersection()** method:

result = A.intersection(B)

print(result)

A real-life example would be students learning two different subjects:

```
python_students = {"Ravi", "Sita", "John"}  
excel_students = {"John", "Meena", "Ravi"}
```

```
both = python_students & excel_students
```

```
print(both)
```

The output is:

```
{"Ravi", "John"}
```

Ravi and John are present in both groups, so they are the intersection.

So, remember **Intersection = common values**.

Difference

Difference means finding the elements that are present in the first set but not present in the second set.

The difference operator is `-`.

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
result = A - B
```

```
print(result)
```

The output is:

```
{1, 2, 3}
```

We are asking: **“What is in A that is not in B?”**

The value **1** is in A but not B, so it is included. The same is true for **2** and **3**. The values **4** and **5** are not included because they are also present in B.

Therefore:

```
A - B = {1, 2, 3}
```

The order of the sets matters in difference. If we reverse them:

```
result = B - A
```

```
print(result)
```

the result becomes:

```
{6, 7, 8}
```

This time we are asking: **“What is in B that is not in A?”**

We can also use the `difference()` method:

```
result = A.difference(B)
```

```
print(result)
```

So, remember **Difference = values in the first set but not in the second set.**

Symmetric Difference

Symmetric difference gives the values that are present in either set but **not in both sets**.

The operator for symmetric difference is `^`.

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
result = A ^ B
```

```
print(result)
```

The output is:

```
{1, 2, 3, 6, 7, 8}
```

To understand this, first identify the common values. The common values are `4` and `5`. Symmetric difference removes these common values. It then keeps the values that belong only to A or only to B. Therefore, `1, 2, 3` from A and `6, 7, 8` from B remain.

We can also use:

```
result = A.symmetric_difference(B)
```

```
print(result)
```

So, remember **Symmetric Difference = everything except the common values.**

Easy Way to Understand All Four Set Operations

Suppose:

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}
```

The **Union** `A | B` takes everything:

```
{1, 2, 3, 4, 5, 6}
```

The **Intersection** $A \& B$ takes only the common values:

$\{3, 4\}$

The **Difference** $A - B$ takes values that are only in A:

$\{1, 2\}$

The **Symmetric Difference** $A \wedge B$ takes values that are not common:

$\{1, 2, 5, 6\}$

An easy way to remember them is:

$|$ → Combine everything

$\&$ → Common values

$-$ → Only from the first set

$\wedge$ → Not common

Practice Program for Set Operations

The following program performs all four operations:

$A = \{10, 20, 30, 40\}$

$B = \{30, 40, 50, 60\}$

```
print("Set A:", A)
```

```
print("Set B:", B)
```

```
print("Union:", A | B)
```

```
print("Intersection:", A & B)
```

```
print("Difference:", A - B)
```

```
print("Symmetric Difference:", A ^ B)
```

The output is:

Set A: $\{10, 20, 30, 40\}$

Set B: $\{30, 40, 50, 60\}$

Union: $\{10, 20, 30, 40, 50, 60\}$

Intersection: $\{30, 40\}$

Difference: $\{10, 20\}$

Symmetric Difference: $\{10, 20, 50, 60\}$

Here, 30 and 40 are common to both sets. Therefore, they appear in the intersection. They are also removed from the symmetric difference. The difference $A - B$ contains only 10 and 20 because those values are in A but not B.

Tuples

A **tuple** is another type of collection in Python. Like a list, a tuple can store multiple values together. The major difference is that a tuple is **immutable**. Immutable means that once a tuple has been created, its existing elements cannot be changed.

Tuples are normally written using **round brackets ()**.

For example:

```
numbers = (10, 20, 30, 40)
```

```
print(numbers)
```

The output is:

```
(10, 20, 30, 40)
```

Here, the variable `numbers` stores four values together inside one tuple.

Characteristics of Tuples

A tuple is **ordered**, which means the values have a fixed sequence. Unlike a set, we can use positions or indexes to access tuple elements.

For example:

```
fruits = ("Apple", "Banana", "Mango")
```

```
print(fruits)
```

The values remain in their given order:

```
("Apple", "Banana", "Mango")
```

A tuple can also contain **duplicate values**. Unlike sets, tuples do not automatically remove duplicates.

```
numbers = (10, 20, 10, 30, 20)
```

```
print(numbers)
```

The output is:

```
(10, 20, 10, 30, 20)
```

Here, `10` occurs twice and `20` occurs twice, and Python keeps all occurrences.

A tuple can also contain **different types of data** in the same collection.

```
student = ("Ravi", 20, 85.5, True)
```

```
print(student)
```

Here, "Ravi" is a string, 20 is an integer, 85.5 is a float, and True is a Boolean value. Python allows these different types to exist together inside one tuple.

Immutability of Tuples

The most important characteristic of a tuple is that it is **immutable**. Once the tuple has been created, we cannot change an existing element.

For example:

```
numbers = (10, 20, 30)
```

```
numbers[0] = 100
```

Here, `numbers[0]` means the first element, which is 10. We are trying to replace 10 with 100.

Python does not allow this because tuples cannot be modified after creation. Therefore, this statement produces a **TypeError**.

This is different from a list.

```
numbers = [10, 20, 30]
```

```
numbers[0] = 100
```

```
print(numbers)
```

The output is:

```
[100, 20, 30]
```

A list allows the value at index 0 to be changed from 10 to 100, but a tuple does not.

An easy real-life way to understand this is to imagine a **printed certificate**. Once the information has been printed, you cannot directly change the printed value. A tuple is similar because its existing elements cannot be modified.

Tuple Indexing

Because tuples are ordered, every element has an index. Python starts indexing from 0, not 1.

Consider:

```
fruits = ("Apple", "Banana", "Mango", "Orange")
```

The positions are:

Apple → 0

Banana → 1

Mango → 2

Orange → 3

Therefore:

```
print(fruits[0])
```

gives:

Apple

and:

```
print(fruits[2])
```

gives:

Mango

The important thing is that the index tells Python **which position we want to access**.

Creating a Tuple

The most common way to create a tuple is by using parentheses:

```
numbers = (10, 20, 30, 40)
```

```
print(numbers)
```

Python also allows us to create a tuple without writing parentheses:

```
numbers = 10, 20, 30, 40
```

```
print(numbers)
```

Python understands that the comma-separated values form a tuple. The output is:

```
(10, 20, 30, 40)
```

The **comma** is actually important in tuple creation.

Creating a Single-Element Tuple

There is a special rule when creating a tuple with only one element. We must put a comma after the element.

For example:

```
number = (10,)

print(number)
print(type(number))
```

The output is:

```
(10,)
<class 'tuple'>
```

The comma after `10` tells Python that this is a tuple.

If we write:

```
number = (10)

print(type(number))
```

the output is:

```
<class 'int'>
```

Why? Because `(10)` is simply the number `10` surrounded by parentheses. The parentheses alone do not make it a tuple. The comma is what makes it a single-element tuple.

Therefore:

```
(10) → integer
(10,) → tuple
```

This is an important point to remember when creating tuples.

Accessing Tuple Elements

Because a tuple has indexes, we can access individual elements using square brackets.

Consider:

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(fruits[0])
print(fruits[2])
```

The first statement asks for the element at index `0`, so Python gives `"Apple"`. The second asks for the element at index `2`, so Python gives `"Mango"`.

The output is:

Apple
Mango

Notice that the tuple itself uses parentheses, but when accessing an element, we use square brackets.

Creating tuple → ()

Accessing tuple → []

Negative Indexing

Python also allows us to access tuple elements from the end using **negative indexes**.

For example:

```
fruits = ("Apple", "Banana", "Mango", "Orange")
```

```
print(fruits[-1])
```

```
print(fruits[-2])
```

The output is:

Orange

Mango

Here, **-1** means the **last element**, and **-2** means the **second-last element**.

The indexing can be understood like this:

Positive:	0	1	2	3
	Apple	Banana	Mango	Orange

Negative:	-4	-3	-2	-1
	Apple	Banana	Mango	Orange

So if we want the last element, instead of counting how many elements are present, we can simply use **-1**.

Slicing a Tuple

Slicing means taking a portion or part of a tuple instead of taking only one element.

The basic syntax is:

```
tuple[start:end]
```

The important rule is that the **start position is included, but the end position is not included**.

For example:

```
numbers = (10, 20, 30, 40, 50)
```

```
print(numbers[1:4])
```

The indexes are:

10 → 0

20 → 1

30 → 2

40 → 3

50 → 4

We are asking for `[1:4]`. Python starts at index **1** and stops **before index 4**. Therefore, indexes **1**, **2**, and **3** are selected.

The output is:

```
(20, 30, 40)
```

A useful way to remember slicing is:

```
[start : end]
```

↑ ↑

include stop before this position

So `[1:4]` means **start at 1 and stop before 4**.

We can also leave the start position empty:

```
numbers = (10, 20, 30, 40, 50)
```

```
print(numbers[:3])
```

This means **start from the beginning and stop before index 3**.

The output is:

```
(10, 20, 30)
```

We can leave the end position empty:

```
print(numbers[2:])
```

This means **start at index 2 and continue until the end**.

The output is:

```
(30, 40, 50)
```

If we write:

```
print(numbers[:])
```

Python takes the complete tuple:

```
(10, 20, 30, 40, 50)
```

These slicing examples are summarized in your source material.

Packing Tuples

Packing means putting multiple individual values into a single tuple.

For example:

```
student = "Ravi", 20, "Computer Science"
```

```
print(student)
```

Python packs these three values into one tuple:

```
("Ravi", 20, "Computer Science")
```

Here, "Ravi", 20, and "Computer Science" were originally separate values. After the assignment, they are stored together inside the `student` tuple.

Another example is:

```
data = 10, 20, 30, 40
```

```
print(data)
```

Python automatically packs the four values into:

```
(10, 20, 30, 40)
```

So, an easy meaning of **packing** is:

Many separate values

↓

One tuple

This is why packing is useful when we want to group related values together.

Unpacking Tuples

Unpacking is the opposite of packing. It means taking the values from one tuple and assigning them to separate variables.

For example:

```
student = ("Ravi", 20, "Computer Science")
```

```
name, age, course = student
```

```
print(name)
```

```
print(age)
```

```
print(course)
```

The tuple contains three values:

```
("Ravi", 20, "Computer Science")
```

Python takes the first value and puts it into `name`. The second value goes into `age`, and the third value goes into `course`.

Therefore:

```
name → "Ravi"
```

```
age → 20
```

```
course → "Computer Science"
```

The output is:

```
Ravi
```

```
20
```

```
Computer Science
```

For normal unpacking, the number of variables should match the number of values in the tuple. If there are three values, we normally provide three variables.

Unpacking Using the Asterisk

Python also provides `*` when unpacking. It can be used when we want one variable to collect multiple values.

For example:

```
numbers = (10, 20, 30, 40, 50)
```

```
first, *middle, last = numbers
```

```
print(first)
```

```
print(middle)
```

```
print(last)
```

Here, `first` gets the first value:

first → 10

last gets the last value:

last → 50

The `*middle` collects all the values between them:

middle → [20, 30, 40]

Therefore, the output is:

10

[20, 30, 40]

50

The `*` is useful when we do not know exactly how many middle values there are or when we want one variable to collect the remaining values.

Tuple Operations

Tuples support several useful operations. We can join tuples, repeat them, check whether a value exists, find their length, count values, find the position of a value, and process each element using a loop.

Concatenation

Concatenation means joining two or more tuples together. The `+` operator is used for this.

A = (1, 2, 3)

B = (4, 5, 6)

C = A + B

print(C)

The output is:

(1, 2, 3, 4, 5, 6)

Here, Python takes all the elements of `A` and then places all the elements of `B` after them. The original tuples are not changed. A new tuple is created and stored in `C`.

Think of it as:

A = (1, 2, 3)

B = (4, 5, 6)

A + B

Output: (1, 2, 3, 4, 5, 6)

Repetition

The `*` operator can be used to repeat the contents of a tuple.

```
numbers = (1, 2, 3)
```

```
result = numbers * 2
```

```
print(result)
```

The output is:

```
(1, 2, 3, 1, 2, 3)
```

Here, `numbers * 2` does not multiply each individual number by 2. Instead, it **repeats the entire tuple two times**.

So:

```
(1, 2, 3) * 2
```

means:

```
(1, 2, 3) + (1, 2, 3)
```

which produces:

```
(1, 2, 3, 1, 2, 3)
```

This is an important difference from mathematical multiplication.

Membership

The `in` operator is used to check whether a particular value exists inside a tuple.

For example:

```
fruits = ("Apple", "Banana", "Mango")
```

```
print("Banana" in fruits)
```

```
print("Orange" in fruits)
```

Python checks whether `"Banana"` exists in the tuple. Since it does, the result is:

```
True
```

Python then checks whether "Orange" exists. It does not, so the result is:

False

Therefore, the complete output is:

True

False

The meaning is simple:

value in tuple → True or False

Length of a Tuple

The `len()` function tells us how many elements are present in a tuple.

For example:

```
numbers = (10, 20, 30, 40, 50)
```

```
print(len(numbers))
```

There are five values in the tuple, so the output is:

5

Notice that `len()` counts the number of elements, while indexing starts from 0.

For example:

10 → index 0

20 → index 1

30 → index 2

40 → index 3

50 → index 4

There are **5 elements**, even though the last index is **4**.

Count

The `count()` method tells us how many times a particular value occurs in a tuple.

For example:

```
numbers = (10, 20, 10, 30, 10)
```

```
print(numbers.count(10))
```

The value 10 appears three times, so the output is:

3

The method does not return the position of the value. It returns **how many times the value occurs**.

So:

```
numbers.count(10)
```

means:

“Count how many times 10 appears in the tuple.”

Index

The `index()` method tells us the position of a particular value.

For example:

```
fruits = ("Apple", "Banana", "Mango", "Orange")
```

```
print(fruits.index("Mango"))
```

The indexes are:

Apple → 0

Banana → 1

Mango → 2

Orange → 3

Therefore, "Mango" is at index 2, and the output is:

2

The important difference is:

`count()` → How many times does it occur?

`index()` → Where does it occur?

Iterating Through a Tuple

We can use a `for` loop to access each element of a tuple one at a time.

```
fruits = ("Apple", "Banana", "Mango")
```

```
for fruit in fruits:
```

```
    print(fruit)
```

The `for` loop takes the first value and puts it into `fruit`, then prints it. It then takes the next value and prints it, continuing until all elements have been processed.

The output is:

Apple
Banana
Mango

This is useful when we want to perform some action on every element of a tuple.

Difference Between Sets and Tuples

The biggest difference between a set and a tuple is how they store and handle data.

A **set** is mainly used when we need unique values and set operations. A **tuple** is mainly used when we want an ordered collection of values that should not be changed.

A set uses `{ }`, while a tuple normally uses `( )`. A set does not support indexing because it is unordered, while a tuple supports indexing because it is ordered. Sets do not allow duplicate values, while tuples allow duplicates. Sets are mutable, meaning we can add or remove elements, while tuples are immutable, meaning their existing elements cannot be changed.

Feature	Set	Tuple
Written using	<code>{ }</code>	<code>()</code>
Order	Unordered	Ordered
Duplicate values	Not allowed	Allowed
Indexing	Not supported	Supported
Slicing	Not supported	Supported
Mutable	Yes	No
Main purpose	Unique values and set operations	Fixed collection of values

Simple Example Using Sets and Tuples

The following program shows both a set and a tuple together:

```
# Set
subjects = {"Python", "Excel", "Python", "Word"}

print("Subjects:", subjects)

# Tuple
student = ("Ravi", 20, "Python")

print("Student:", student)

# Accessing tuple
print("Name:", student[0])
print("Age:", student[1])
print("Course:", student[2])
```

First, the `subjects` set is created. `"Python"` is written twice, but because a set does not allow duplicates, Python keeps only one `"Python"`.

The `student` tuple contains three values: the student's name, age, and course. Since a tuple is ordered, we can access these values using indexes. `student[0]` gives the name, `student[1]` gives the age, and `student[2]` gives the course.

The output is:

```
Subjects: {'Python', 'Excel', 'Word'}
Student: ('Ravi', 20, 'Python')
Name: Ravi
Age: 20
Course: Python
```

This example demonstrates the main difference very clearly: the **set removes the duplicate `"Python"`**, while the **tuple keeps its values in order and allows us to access them using indexes**.

SET → Unique values

TUPLE → Ordered, fixed values.